Trabalho Prático 1 — Projeto e Análise de Algoritmos

Parte da Pessoa 1: leitura de entrada, estruturas de dados e gerador de mapas

Disciplina: Projeto e Análise de Algoritmos — UFV, Campus Florestal — 2026/2
Escopo deste texto: subsídios para compor o relatório do grupo. A implementação do backtracking, a análise de sua complexidade, a integração do programa principal e os resultados globais deverão ser incorporados pelos outros integrantes.

1. Formato do arquivo de entrada

O programa recebe o labirinto por meio de um arquivo de texto. A primeira linha possui dois inteiros positivos que especificam, nesta ordem, o número de linhas e o número de colunas. A segunda linha informa a quantidade de chaves necessárias para abrir o baú. Em seguida, cada uma das linhas do arquivo representa uma linha do labirinto. São aceitos os seguintes símbolos: `A` (posição inicial do aventureiro), `C` (chave), `X` (baú), `0` (sala livre) e `1` (parede). O leitor aceita símbolos adjacentes ou separados por espaços em uma mesma linha, como ilustrado no enunciado. As coordenadas são indexadas a partir de zero.

Exemplo de entrada com quatro linhas, cinco colunas e duas chaves:

```
4 5
2
A01C0
00010
01000
0C00X
```

2. Estruturas de dados

A implementação foi desenvolvida em linguagem C, separada nos arquivos `labirinto.h` (tipos e protótipos) e `labirinto.c` (funções). O tipo `Posicao` armazena os campos `linha` e `coluna`, ambos do tipo `size_t`. O tipo `Labirinto` contém `linhas`, `colunas`, `quantidade_chaves`, `char **celulas`, `Posicao inicio`, `Posicao bau` e `Posicao *chaves`.

A matriz dinâmica `celulas` armazena o símbolo original de cada sala em `celulas[i][j]`. Os registros de início e baú permitem localizar rapidamente as posições especiais, enquanto o vetor de chaves registra as coordenadas de todas as células marcadas com `C`. A representação conserva os símbolos de entrada para que o módulo de busca possa decidir movimentos e contabilizar as chaves sem depender de um formato de arquivo específico.

3. Leitura, validação e memória

A função pública `labirinto_carregar` abre o arquivo, lê as dimensões e a quantidade de chaves, aloca a matriz, percorre cada linha e identifica as posições de `A`, `C` e `X`. O leitor rejeita dimensões não positivas, quantidade incompatível de chaves, caracteres não permitidos, linhas com número incorreto de colunas, entradas incompletas, dados extras, ausência ou multiplicidade das posições de início e baú, ou divergência entre a quantidade declarada de chaves e o número de símbolos `C` encontrados. Em caso de falha, é retornado um código de erro e, quando disponível, uma mensagem descritiva.

A alocação é dinâmica porque as dimensões são determinadas durante a execução. A estrutura é alocada com `calloc`; o vetor de ponteiros das linhas é alocado dinamicamente; cada linha recebe memória suficiente para suas células e um caractere terminador; o vetor de chaves é alocado conforme a quantidade declarada. Em qualquer falha intermediária, o leitor desaloca as partes que já tenham sido criadas. A função `labirinto_liberar` centraliza a liberação de linhas, matriz, vetor de chaves e estrutura principal.

Duas funções auxiliares compõem a interface: `labirinto_em_limites`, que informa se uma coordenada pertence à matriz, e `labirinto_eh_transitavel`, que informa se a posição é válida e não contém uma parede. O controle de salas visitadas não faz parte da leitura; ele é responsabilidade do algoritmo de backtracking.

4. Modularização e integração

O módulo de entrada não possui uma função `main`, podendo ser vinculado ao programa principal desenvolvido pelo integrante responsável pela integração. O contrato da interface expõe `Labirinto` e `Posicao`, com a função `labirinto_carregar(const char *, Labirinto **, char *, size_t)` para carregar o mapa e `labirinto_liberar(Labirinto *)` para liberar a memória. O código de busca poderá consultar `lab->celulas`, `lab->inicio`, `lab->bau`, `lab->chaves` e `lab->quantidade_chaves` sem implementar nova leitura de arquivos.

A decisão por essa separação reduz o acoplamento entre o tratamento da entrada e a busca recursiva. A validação do conteúdo é centralizada, mas a existência de um caminho válido **não** é avaliada pelo leitor, pois depende do algoritmo de busca implementado por outro integrante.

5. Extra opcional: gerador de mapas

Foi desenvolvido o programa independente `gerador_mapas.c`, também em linguagem C. Ele recebe parâmetros de linha de comando para configurar o número de linhas (`--linhas`), colunas (`--colunas`), chaves (`--chaves`), paredes (`--paredes`) e o caminho do arquivo de saída (`--saida`). O parâmetro `--semente` permite repetir a mesma sequência de geração. A quantidade de paredes é **exata**, e não uma porcentagem.

No modo aleatório, as posições são embaralhadas, e as paredes, início, baú e chaves são distribuídos de acordo com as quantidades solicitadas. Esse modo pode produzir mapas com ou sem solução, adequados a diferentes testes. Com `--garantir-solucao`, é reservado um percurso serpentino sem células repetidas, colocando-se nele início, baú e todas as chaves; as paredes são inseridas fora desse percurso. Assim, o mapa gerado possui pelo menos uma solução, respeitando a restrição de coleta de chaves e movimentação ortogonal. O arquivo produzido emprega exatamente o formato aceito pelo leitor, com duas linhas de cabeçalho e a matriz de símbolos. Não foi necessária outra linguagem para o extra.

Exemplo de geração com solução garantida:

```
gcc -std=c11 -Wall -Wextra -Wpedantic extras/gerador_mapas.c -o gerador_mapas
./gerador_mapas --linhas 8 --colunas 10 --chaves 3 \
  --paredes 30 --saida mapa.txt --semente 42 --garantir-solucao
```

6. Testes do módulo

Os testes de leitura foram implementados em `teste_leitura.c` e executados separadamente do programa principal. Foram utilizados os seguintes casos: arquivo do exemplo oficial; linhas com espaços entre símbolos; mapa sem chaves; mapa sintaticamente válido sem caminho possível; símbolo proibido; ausência de baú; falta de chave declarada; linha com colunas excedentes; dois inícios; conteúdo adicional depois da matriz; dimensão inválida; mapa incompleto; e arquivo inexistente.

Resultado observado na execução local: 13 testes de leitura executados, **13 aprovados**. A compilação foi realizada com GCC em C11 e com as opções `-Wall -Wextra -Wpedantic -Werror`, sem avisos. Os arquivos produzidos pelo gerador foram lidos com sucesso, e testes adicionais confirmaram as contagens exatas de dimensões, chaves e paredes. No modo com solução garantida, um percurso serpentino foi verificado coletando todas as chaves até o baú.

Esses resultados verificam a entrada, as estruturas e o gerador; não são resultados de desempenho do algoritmo de backtracking, que deverá ser testado após a integração.

7. Considerações sobre a integração e o relatório final

Os arquivos devem ser integrados ao projeto da equipe com o Makefile e o programa principal. A documentação final do grupo deverá incluir a explicação da busca recursiva, análise de complexidade de tempo e espaço, modo normal e modo análise, resultados completos de execução (inclusive total de chamadas recursivas e nível máximo de recursão), arquivos de entrada, e demais requisitos e diretrizes de entrega definidos pelo professor.

Observação de consistência do enunciado: a saída ilustrativa contém algumas coordenadas repetidas, embora a descrição textual proíba revisitar a mesma sala no caminho. Recomenda-se que o grupo confirme a interpretação com o professor se necessário; o módulo de entrada não faz suposições sobre esse detalhe da busca.